Securitize Solana SPL ACL Audit Report

Prepared by Cyfrin

Version 2.0

Lead Auditors

Farouk

JesJupyter

Ctrus

March 17, 2026

Contents

1 About Cyfrin **2**

2 Disclaimer **2**

3 Risk Classification **2**

4 Protocol Summary **2**

5 Audit Scope **2**

6 Executive Summary **3**

7 Findings **6**

7.1 Medium Risk . . . 6

7.1.1 Permissionless freeze setup is wired to the thaw extra-metas PDA instead of the freeze PDA 6

7.2 Low Risk . . . 7

7.2.1 `setup_extra_metas` forwards PDA signer authority to an unvalidated `gating_program` in an admin-only path . . . 7

7.2.2 `thaw_permissionless` and `freeze_permissionless` Can Be Bypassed by Direct `token_acl` Invocation . . . 7

7.3 Informational . . . 9

7.3.1 `change_admin` is single-step and permits irreversible admin loss through bad input . . . 9

7.3.2 `token_account` is missing a local `mint == mint.key()` constraint in authority flows . . . 9

7.3.3 Several error variants are defined but currently unused . . . 9

7.3.4 Incorrect Documentation Comment for `create_config_handler` . . . 9

7.3.5 `new_freeze_authority` Omitted from `DeleteConfig` Event . . . 10

7.3.6 Unnecessary mut constraint on mint in read-only ops delays genuine operations that require mint to be mutable . . . 10

7.4 Gas Optimization . . . 12

7.4.1 Redundant executable check for token ACL program after address validation . . . 12

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Securitize SPL Token Access Control is a Solana Anchor program that interposes a role-based access control layer between Token-2022 mints and their mint and freeze authority operations. A designated admin initializes the program for a given mint, after which the program's custody PDA (`access_control_authority`) holds mint and freeze authority on that Token-2022 mint. The admin manages up to ten freeze authorities and ten mint authorities per mint, and any registered authority can invoke the corresponding token operation — minting, freezing, or thawing — through the program, which signs the downstream CPI call on their behalf.

The program supports two distinct freeze authority modes. In the direct mode, the custody PDA holds freeze authority on the Token-2022 mint and signs freeze and thaw CPIs to Token-2022. In the delegated mode, freeze authority is transferred to a Token-ACL (sRFC-37) mint config account, and the program interacts with the Token-ACL program to freeze and thaw accounts. The delegated mode also enables permissionless freeze and thaw, where any signer can trigger the operation provided an external gating program authorizes the call. The admin configures the gating program, sets up extra account metadata, and toggles permissionless access through dedicated sRFC-37 management instructions.

The architecture centers on two PDAs per mint: `access_control_state`, which stores the admin address, authority lists, and pause flag, and `access_control_authority`, which acts as the signing authority for all CPI calls. The program makes external calls to Token-2022 for token operations and to Token-ACL for sRFC-37 configuration and permissionless flows. When permissionless mode is active, Token-ACL delegates authorization to a configurable gating program, which serves as the final arbiter of whether a freeze or thaw should proceed. Initialization is restricted to a hardcoded allowlist of six Securitize-controlled public keys in production builds, with a devnet feature flag that bypasses this restriction for testing.

For the system to remain secure, the admin key must be protected and operated honestly, the Token-ACL and gating programs must behave correctly, and the devnet feature flag must never be enabled in production builds.

5 Audit Scope

The audit scope was limited to:

```
programs/spl-token-access-control/src/instructions/admin/add_authority.rs
programs/spl-token-access-control/src/instructions/admin/change_admin.rs
programs/spl-token-access-control/src/instructions/admin/initialize.rs
programs/spl-token-access-control/src/instructions/admin/pause.rs
programs/spl-token-access-control/src/instructions/admin/remove_authority.rs
programs/spl-token-access-control/src/instructions/admin/set_authority.rs
programs/spl-token-access-control/src/instructions/admin/unpause.rs
programs/spl-token-access-control/src/instructions/authority/freeze_account.rs
programs/spl-token-access-control/src/instructions/authority/mint_to_checked.rs
programs/spl-token-access-control/src/instructions/authority/thaw_account.rs
programs/spl-token-access-control/src/instructions/cpi/srfc_37/create_config.rs
programs/spl-token-access-control/src/instructions/cpi/srfc_37/delete_config.rs
programs/spl-token-access-control/src/instructions/cpi/srfc_37/freeze_account.rs
programs/spl-token-access-control/src/instructions/cpi/srfc_37/freeze_permissionless.rs
programs/spl-token-access-control/src/instructions/cpi/srfc_37/set_authority.rs
programs/spl-token-access-control/src/instructions/cpi/srfc_37/set_gating_program.rs
programs/spl-token-access-control/src/instructions/cpi/srfc_37/setup_extra_metas.rs
programs/spl-token-access-control/src/instructions/cpi/srfc_37/thaw_account.rs
programs/spl-token-access-control/src/instructions/cpi/srfc_37/thaw_permissionless.rs
programs/spl-token-access-control/src/instructions/cpi/srfc_37/toggle_permissionless.rs
programs/spl-token-access-control/src/instructions/cpi/token_2022/freeze_account.rs
programs/spl-token-access-control/src/instructions/cpi/token_2022/mint_to_checked.rs
programs/spl-token-access-control/src/instructions/cpi/token_2022/set_authority.rs
programs/spl-token-access-control/src/instructions/cpi/token_2022/thaw_account.rs
programs/spl-token-access-control/src/instructions/srfc_37/create_config.rs
programs/spl-token-access-control/src/instructions/srfc_37/delete_config.rs
programs/spl-token-access-control/src/instructions/srfc_37/freeze_permissionless.rs
programs/spl-token-access-control/src/instructions/srfc_37/set_gating_program.rs
programs/spl-token-access-control/src/instructions/srfc_37/setup_extra_metas.rs
programs/spl-token-access-control/src/instructions/srfc_37/thaw_permissionless.rs
programs/spl-token-access-control/src/instructions/srfc_37/toggle_permissionless.rs
programs/spl-token-access-control/src/states/access_control_state.rs
programs/spl-token-access-control/src/constants.rs
programs/spl-token-access-control/src/discriminators.rs
programs/spl-token-access-control/src/lib.rs
```

6 Executive Summary

Over the course of 3 days, the Cyfrin team conducted an audit on the Securitize Solana SPL ACL smart contracts provided by Securitize. In this period, a total of 10 issues were found.

Securitize SPL Token Access Control is a Solana Anchor program that wraps Token-2022 mint and freeze authority behind a role-based access control layer, supporting both direct Token-2022 authority and delegated authority via the sRFC-37 Token-ACL program for permissionless, gating-program-controlled freeze and thaw operations. The security model relies on correct PDA custody of mint and freeze authorities, proper enforcement of authority membership and pause state, and the integrity of the external Token-ACL and gating programs.

Our review focused on CPI call correctness across both the Token-2022 and sRFC-37 paths, access control enforcement on all authority-gated and admin-gated instructions, the permissionless freeze and thaw security model, authority management edge cases, PDA derivation correctness, and event emission completeness. The audit identified 10 findings: 1 Medium, 2 Low, 5 Informational, and 1 Gas Optimization. No critical vulnerabilities were discovered.

The medium severity finding relates to the permissionless freeze extra-metas setup, which is wired to the thaw-side PDA seed rather than the freeze-side PDA seed, meaning permissionless freeze may fail at runtime for gating programs that depend on freeze-specific extra account metadata. The low severity items address a pause bypass vector where users can invoke Token-ACL directly to circumvent the program's pause check on permissionless operations, and an admin safety concern where `setup_extra_metas` forwards PDA signer authority to an unvalidated

gating program. Informational findings cover defense-in-depth improvements such as adding a two-step admin transfer pattern, enforcing token account mint constraints locally, removing unused error variants, correcting a misleading doc comment, and including the new freeze authority in the `DeleteConfig` event. The gas optimization item identifies redundant executable checks on the Token-ACL program after address validation.

Overall, the codebase demonstrates a solid security foundation with well-structured access controls, clear separation between admin and authority roles, and correct PDA derivation.

Summary

Project Name	Securitize Solana SPL ACL
Repository	bc-solana-spl-acl-sc
Commit	df60662bbc47...
Audit Timeline	Mar 10th - Mar 12th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	1
Low Risk	2
Informational	6
Gas Optimizations	1
Total Issues	10

Summary of Findings

[M-1] Permissionless freeze setup is wired to the thaw extra-metas PDA instead of the freeze PDA	Resolved
[L-1] `setup_extra_metas` forwards PDA signer authority to an unvalidated `gating_program` in an admin-only path	Resolved
[L-2] `thaw_permissionless` and `freeze_permissionless` Can Be Bypassed by Direct `token_acl` Invocation	Resolved
[I-1] `change_admin` is single-step and permits irreversible admin loss through bad input	Acknowledged
[I-2] `token_account` is missing a local `mint == mint.key()` constraint in authority flows	Resolved
[I-3] Several error variants are defined but currently unused	Resolved
[I-4] Incorrect Documentation Comment for `create_config_handler`	Resolved
[I-5] `new_freeze_authority` Omitted from `DeleteConfig` Event	Resolved

[I-6] Unnecessary mut constraint on mint in read-only ops delays genuine operations that require mint to be mutable	Resolved
[G-1] Redundant executable check for token ACL program after address validation	Resolved

7 Findings

7.1 Medium Risk

7.1.1 Permissionless freeze setup is wired to the thaw extra-metas PDA instead of the freeze PDA

Description: The program exposes only one extra-metas seed, `thaw_extra_account_metas`, in `constants.rs`, and `setup_extra_metas` can only initialize a PDA derived from that thaw seed. The local SDK mirrors that assumption: `extraMetasPda()` always derives `["thaw_extra_account_metas", mint]`, and the `freezePermissionless` builder passes that same PDA in the remaining accounts for permissionless freeze.

```rust
/// CHECK: Extra metas pda account checking in gating program
#[account(
    mut,
    seeds = [constants::THAW_EXTRA_ACCOUNT_METAS_SEED, mint.key().as_ref()],
    bump,
    seeds::program = gating_program.key()
)]
pub extra_metas: AccountInfo<'info>,
```

```rust
#[constant]
pub const THAW_EXTRA_ACCOUNT_METAS_SEED: &[u8] = b"thaw_extra_account_metas";
```

However, the vendored Token-ACL interface and SDK clearly model two distinct validation accounts:

- thaw uses `["thaw_extra_account_metas", mint]`
- freeze uses `["freeze_extra_account_metas", mint]`

This is not just a naming difference in the client code. Token-ACL's on-chain `invoke_can_freeze_permissionless` helper computes the freeze-side PDA with `get_freeze_extra_account_metas_address(...)` and only loads extra account metadata if that exact freeze PDA is present in `additional_accounts`. If the caller instead supplies the thaw PDA, the freeze helper does not treat it as the validation account and does not resolve the freeze-side extra account dependencies from it.

As a result, the current integration path is miswired:

1. `setup_extra_metas` only prepares the thaw-side validation account.
2. `freezePermissionless` still forwards that thaw-side account during freeze.
3. Token-ACL freeze resolution expects the freeze-side account and ignores the thaw-side one for dependency expansion.

Impact: Permissionless thaw can be configured correctly, but permissionless freeze is only reliable for trivial gate programs that require no freeze-side extra metas. Any gate that follows the standard freeze interface and depends on `freeze_extra_account_metas` for account resolution will fail to receive its expected dependency set and may revert at runtime.

Recommended Mitigation: Add a distinct freeze extra-metas seed and setup flow on the on-chain side, then update the SDK so:

- thaw derives and supplies `thaw_extra_account_metas`
- freeze derives and supplies `freeze_extra_account_metas`

Securitize: Fixed in 4794d46.

Cyfrin: Verified.

7.2 Low Risk

7.2.1 `setup_extra_metas` forwards PDA signer authority to an unvalidated `gating_program` in an admin-only path

Description: `setup_extra_metas` only checks that the supplied `gating_program` is executable. It does not verify that the target program matches the gate configured for the mint. The CPI helper then invokes that executable with `invoke_signed`, forwarding the `access_control_authority` PDA as a signer-capable account. This path is explicitly admin-only. Under the stated trust model, the admin is already trusted and can configure the gating program elsewhere in the protocol. As a result, this is better characterized as an admin safety / integration-hardening issue than a direct privilege-escalation issue.

```
/// CHECK: Gating program
#[account(constraint = gating_program.executable @ AccessControlError::NotExecutable)]
pub gating_program: AccountInfo<'info>,
```

```
let ix = Instruction {
    program_id: ctx.accounts.gating_program.key(),
    accounts: accounts_meta,
    data: instruction_data,
};

let mut account_infos = vec![
    ctx.accounts.access_control_authority.to_account_info(),
    ctx.accounts.admin.to_account_info(),
    ctx.accounts.mint_config.to_account_info(),
    ctx.accounts.mint.to_account_info(),
    ctx.accounts.extra_metas.to_account_info(),
    ctx.accounts.system_program.to_account_info(),
];

account_infos.extend_from_slice(ctx.remaining_accounts);

invoke_signed(&ix, &account_infos, signer_seeds)?;
```

Impact: The main risk is misconfiguration or malicious frontend / operator confusion: an admin can be tricked into calling `setup_extra_metas` against the wrong executable and unintentionally forward signer privileges into an unexpected CPI target.

Recommended Mitigation: Before the CPI, deserialize the relevant Token-ACL config and require the supplied `gating_program` to equal the configured gate for the mint.

Securitize: Fixed in 0330e59.

Cyfrin: Verified.

7.2.2 `thaw_permissionless` and `freeze_permissionless` Can Be Bypassed by Direct `token_acl` Invocation

Description: The `is_paused` check in `thaw_permissionless_handler` and `freeze_permissionless_handler` only applies when users invoke the spl-token-access-control program. Users can bypass this check by calling the `token_acl` program directly, since `token_acl` does not enforce the ACL program's pause state. The admin's ability to pause permissionless thaw/freeze operations is therefore ineffective for direct `token_acl` callers.

The ACL program checks `AccessControlState.is_paused` before CPI-ing to token_acl:

```
    require!(
        !ctx.accounts.access_control_state.is_paused,
        AccessControlError::Paused
    );
```

The same check exists in `freeze_permissionless_handler`.

However, the `token_acl` program is a separate, publicly callable program. Users can construct transactions that invoke token_acl's `thaw_permissionless` (or `freeze_permissionless`) instruction directly, without going through `spl-token-access-control`. The `token_acl` program does not read or validate `AccessControlState` as it only invokes the gating program for its own checks.

Impact: When the admin pauses the ACL program, they may expect all permissionless thaw/freeze operations to stop. In reality, users who call `token_acl` directly can still thaw or freeze accounts.

Recommended Mitigation: If the team intends pause to apply to all permissionless thaw/freeze operations, the check must be enforced in a place that cannot be bypassed. The `token_acl` program invokes the gating program for its gating logic. The gating program is the single point through which all thaw/freeze permissionless flows pass.

Securitize: Fixed in d041c66.

Cyfrin: Verified.

7.3 Informational

7.3.1 `change_admin` is single-step and permits irreversible admin loss through bad input

Description: `change_admin` immediately overwrites the stored admin with the supplied `new_admin`. There is no acceptance step, and there is no guard against obviously bad destinations such as the default pubkey or `system_program::ID`.

This is not an exploit against an honest admin, but it is a real operational footgun: a typo or bad integration call can permanently brick future admin operations.

Recommended Mitigation: Use a two-step transfer pattern with `pending_admin` plus an explicit `accept_admin` flow, and reject sentinel addresses such as `Pubkey::default()` and `system_program::ID`.

Securitize: Acknowledged.

7.3.2 `token_account` is missing a local `mint == mint.key()` constraint in authority flows

Description: The authority's `freeze_account`, `thaw_account`, and the `mint_to_checked` instructions do not locally enforce that the provided token account belongs to the supplied mint. Downstream Token-2022 or Token-ACL checks should still reject mismatches, so the issue is primarily defense-in-depth and error-surface quality rather than a direct exploit.

Recommended Mitigation: Add an explicit local `token_account.mint == mint.key()` constraint to each affected instruction context.

Securitize: Fixed in f65f393.

Cyfrin: Verified.

7.3.3 Several error variants are defined but currently unused

Description: The error enum currently contains several variants that appear unused across the program logic and tests, including `SameAuthority`, `InvalidProgram`, `InvalidMint`, `InsufficientBalance`, `MissingMintConfig`, `MissingTokenAcl`, and `InvalidMintConfig`.

The clearest example is `SameAuthority`: `set_authority` does not use it to reject no-op transfers where the new authority equals the current authority. The other variants look like dead-code placeholders or validation branches that were planned but never wired into instruction handlers.

Recommended Mitigation: Either wire these variants into real validation paths or remove them so the error surface matches actual program behavior.

Securitize: Fixed in 29df9ca.

Cyfrin: Verified.

7.3.4 Incorrect Documentation Comment for `create_config_handler`

Description: The doc comment for `create_config_handler` incorrectly describes the behavior of `delete_config_handler`. The comment states "Delete mint config and sent lamports to receiver" when the function actually creates a mint config via CPI to the token-acl program.

```
/// Delete mint config and sent lamports to receiver
pub fn create_config_handler<'info>(
    ctx: Context<'_, '_, '_, 'info, CreateConfig<'info>>,
) -> Result<()> {
    cpi::srfc_37::create_config::handler(&ctx)?;

    emit_cpi!(events::CreateConfig {
        access_control_state_key: ctx.accounts.access_control_state.key(),
        admin: ctx.accounts.admin.key(),
        mint: ctx.accounts.mint.key(),
```

```
        mint_config: ctx.accounts.mint_config.key(),
    });

    Ok(())
}
```

Impact: Developers may misinterpret the function's purpose, leading to confusion and potential misuse during future development.

Recommended Mitigation: Update the doc comment to accurately describe the function's behavior.

Securitize: Fixed in 2c439f9.

Cyfrin: Verified.

7.3.5 `new_freeze_authority` Omitted from `DeleteConfig` Event

Description: The `DeleteConfig` event does not include the `new_freeze_authority` field, even though the instruction transfers the mint's freeze authority to this account. This creates an incomplete audit trail and prevents off-chain indexers from knowing who holds freeze authority after a config deletion.

```
/// Delete mint config and sent lamports to receiver
pub fn delete_config_handler<'info>(
    ctx: Context<'_, '_, '_, 'info, DeleteConfig<'info>>,
) -> Result<()> {
    cpi::srfc_37::delete_config::handler(&ctx)?;

    emit_cpi!(events::DeleteConfig {
        access_control_state_key: ctx.accounts.access_control_state.key(),
        admin: ctx.accounts.admin.key(),
        mint: ctx.accounts.mint.key(),
        mint_config: ctx.accounts.mint_config.key(),
        receiver: ctx.accounts.receiver.key(),
    });

    Ok(())
}
```

By contrast, `SetFreezeAuthority` and `SetMintAuthority` events include both old_authority and new_authority, establishing the pattern that authority changes should be logged.

Impact: Audit trail: Off-chain systems cannot determine who holds freeze authority after a `DeleteConfig` without parsing on-chain account state. Indexing: Indexers and analytics tools cannot fully reconstruct authority history from events alone.

Recommended Mitigation: Add `new_freeze_authority` to the `DeleteConfig` event struct.

Securitize: Fixed in 0fbfe5d.

Cyfrin: Verified.

7.3.6 Unnecessary mut constraint on mint in read-only ops delays genuine operations that require mint to be mutable

Description: Several instructions mark the `mint` account as mutable when they only read from it, wasting compute on unnecessary write-lock acquisition. While the cu itself may not be the problem, but users trying to interact with instructions that actually modifies mint's state and hence require mint to be mutably passed may face delays for no reasons because once an account is passed as mutable in one instruction, it can't be passed as mutable in another until the prev instruction goes by. for eg. `mint_to_checked` instruction requires mint account to be mutable because it is genuinely modifying mint state, but `set_gating_program` is not modifying any state on mint and only needs it for validation, it makes sense to pass mint account as readable here so that other instructions like `mint_to_checked` does not face delays.

Impact: Unnecessary delay on genuine ops that actually modify mint.

Recommended Mitigation: Remove the `mut` constraint from mint. Here are the instances:

1. set_gating_program.rs
2. toggle_permissionless.rs

Securitize: Fixed in 5d87019.

Cyfrin: Verified.

7.4 Gas Optimization

7.4.1 Redundant executable check for token ACL program after address validation

Description: Throughout the codebase, when validating the Token ACL program account, the code performs both an address validation check against the hardcoded `TOKEN_ACL` constant and an `executable` check. Once the address is verified to match a known deployed program ID, the executable check becomes redundant and wastes compute units.

A deployed program on Solana at a specific address is inherently executable - the address validation alone is sufficient to ensure the account is the correct Token ACL program

Impact: Each redundant `executable` check consumes extra compute units which is completely unnecessary because it does not add extra security.

Recommended Mitigation: The address check against `TOKEN_ACL` constant is sufficient since a deployed program at that address must be executable, remove redundant `executable` check from all places. Here are the instances:

1. create_config.rs
2. delete_config.rs
3. freeze_permissionless.rs
4. set_gating_program.rs
5. thaw_permissionless.rs
6. toggle_permissionless.rs
7. freeze_account.rs
8. thaw_account.rs
9. set_authority.rs

Securitize: Fixed in 40ed16f.

Cyfrin: Verified.